

# Python — Otázky a Odpovede

Príprava na skúšku / písomku

## ZÁKLADY

---

### 1. Čo je objekt v OOP?

Objekt **je** jednotka, ktorá obsahuje dáta (atribúty) aj spôsoby na ich manipuláciu (metódy). Je to konkrétna inštancia triedy – trieda je šablona, objekt je vytvorená vec podľa nej.

```
auto = Auto() # auto je objekt
```

### 2. Akými spôsobmi vieme spustiť skript v Pythone?

1. **V termináli:** python script.py
2. **Cez IDE** (PyCharm, VS Code) — tlačidlo Run
3. **Interaktívny mód:** napíšeš python a píšeš priamo
4. **Cez shebang** na Linuxe/Macu: ./script.py
5. **Ako modul:** python -m modul

### 3. Aký je rozdiel medzi programom a skriptom?

**Program** — väčší, samostatný celok (často s GUI alebo komplexnou štruktúrou).

**Skript** — menší kód, zvyčajne na automatizáciu konkrétnej úlohy, spúšťaný interpretrom. V praxi je hranica rozmazaná — rozdiel je skôr vo veľkosti a účele, nie v tom, či je kód kompilovaný.

### 4. Aký je rozdiel medzi kompilátorom a interpretrom?

**Kompilátor:** preloží celý kód do strojového jazyka naraz (C, C++). Výsledok je spustiteľný súbor.

**Interpreter:** vykonáva kód riadok po riadku za behu (Python). Nie je potrebný preklad vopred.

### 5. Čo je shebang a načo ho využiješ v script.py?

Prvý riadok súboru. Hovorí OS, akým interpretrom má súbor spustiť. Využiješ ho na Linuxe/Mac, keď chceš spúšťať skript priamo (./script.py).

```
#!/usr/bin/env python3
```

### 6. Čo je algoritmus?

Presný postup krokov na riešenie problému. Musí byť konečný, jednoznačný a viesť k výsledku.

Príklad: recept na jedlo je algoritmus.

## 7. Čo je cyklus, aké cykly poznáme, čo je iterácia?

Cyklus opakuje blok kódu.

for — iteruje cez sekvenciu (list, range...)

while — opakuje kým platí podmienka

Iterácia = jedno prebehnutie cyklu (jeden krok opakovania).

## 8. Čo je iterácia?

Jedno prebehnutie tela cyklu. Ak má for cyklus 5 prvkov, prebehne 5 iterácií.

## 9. Popíš geometrické útvary vo vývojovom diagrame.

**Ovál / zaoblený obdĺžnik** — Štart / Koniec

**Obdĺžnik** — Proces / akcia

**Kosoštvorec** — Rozhodnutie (áno/nie)

**Rovnobežník** — Vstup / výstup (I/O)

**Šípky** — Tok programu

## 10. Na čo slúži interpretér a ako sa využíva napr. v PyCharm?

Interpreter prekladá a vykonáva kód riadok po riadku.

V PyCharm ho využiješ pri spustení skriptu (Run), v integrovanom termináli a pri debugovaní — vždy beží kód cez zvolený interpreter.

Každý projekt môže mať nastavený iný interpreter (rôzne verzie Pythonu alebo venv).

## OOP

### 11. Čo je abstrakcia v OOP?

Skrývanie nepotrebných detailov, zobrazenie len toho podstatného.

Napr. metóda .drive() — nemusíš vedieť, ako funguje motor, len že auto jazdí.

### 12. Čo je dedičnosť v OOP?

Trieda môže zdediť dáta a metódy inej triedy.

Pes zdedí všetko zo Zviera a môže pridať vlastné veci.

```
class Pes(Zviera):  
    pass
```

### 13. Čo je polymorfizmus v OOP?

Rôzne objekty môžu reagovať na rovnakú metódu inak. Napr. .speak() na Psovi vypíše 'Hav', na Mačke 'Mňau'. Rovnaké volanie, rôzne správanie.

## 14. Ako vytvárame objekt z triedy?

Zavoláme triedu ako funkciu, Python automaticky spustí `__init__`.

```
class Auto:
    def __init__(self, znacka):
        self.znacka = znacka

auto = Auto('BMW')
```

## 15. Ako overím, či bol objekt vytvorený z danej triedy?

**isinstance()** — funguje aj s dedičnosťou (True aj pre podtriedy).

**type()** — prísnejšie: vráti True len pre presnú triedu.

```
isinstance(auto, Auto)    # True
type(auto) == Auto       # True (prísne)
```

## 16. Čo je konštruktor? (`__init__`)

Metóda `__init__` sa automaticky zavolá pri vytvorení objektu. Inicializuje jeho atribúty. Volá sa raz — pri vytvorení objektu.

```
class Osoba:
    def __init__(self, meno):
        self.meno = meno # nastaví atribút
```

## 17. Čo sú dunder metódy?

Metódy s dvojitém podčiarkovníkom (`__init__`, `__str__`, `__repr__`, `__len__`, `__eq__`...).

Python ich volá automaticky v určitých situáciách:

- `__str__` sa zavolá pri `print(obj)`
- `__len__` pri `len(obj)`
- `__eq__` pri `obj1 == obj2`

## FUNKCIE

## 18. Na čo slúži funkcia `type()` a kedy ju využiješ?

Vráti dátový typ hodnoty. Využiješ pri debugovaní či keď kontroluješ vstup.

```
type(42)          # <class 'int'>
type('ahoj')     # <class 'str'>
```

## 19. Na čo slúži funkcia?

Blok kódu, ktorý má názov a môžeš ho opakovane volať.

Zabraňuje opakovaniu kódu, sprehľadňuje program a umožňuje testovanie po častiach.

## 20. Aký je rozdiel medzi return, exit a yield?

**return** — vráti hodnotu z funkcie a ukončí ju.

**exit()** — ukončí celý program.

**yield** — vráti hodnotu z generátora, ale funkcia nekončí — pri ďalšom volaní pokračuje ďalej.

## 21. Čo sú lambda funkcie?

Skrátený zápis funkcie na jeden riadok.

Syntax: `lambda parametre: výraz` — žiadne `def`, žiadne `return`.

Používa sa tam, kde potrebuješ jednorazovú funkciu priamo na mieste.

```
# normalna funkcia
def zdvoj(x):
    return x * 2

# to iste ako lambda
zdvoj = lambda x: x * 2

# typicke pouzitie - priamo na mieste
sorted(ludia, key=lambda x: x['vek'])
```

## 22. Kde a ako použiješ filter() a map()?

Obe prechádzajú cez list a niečo robia s každým prvkom.

**map()** — aplikuje funkciu na každý prvok a vráti upravené hodnoty.

**filter()** — ponechá len prvky, kde je podmienka True.

```
nums = [1,2,3,4,5]
parné = list(filter(lambda x: x%2==0, nums)) # [2,4]
dvojité = list(map(lambda x: x*2, nums))     # [2,4,6,8,10]
```

## 23. Ako funguje any() a all()? Uved' príklad.

**any()** — True ak aspoň jeden prvok je True.

**all()** — True ak všetky prvky sú True.

```
any([False, False, True]) # True
all([True, True, False])  # False
any(x > 0 for x in [-1, 2]) # True
```

---

## DÁTOVÉ TYPY

### 24. Funkcie/metódy objektu STRING — vymenuj základné.

`.upper()`, `.lower()` — zmena veľkosti  
`.strip()` — odstráni biele znaky z okrajov  
`.split(sep)` — rozdelí na list  
`.replace(old, new)` — náhrada  
`.find(s)` — index výskytu, -1 ak nenájde  
`.startswith()`, `.endswith()`  
`.join(list)` — spojí list do stringu  
`len(s)` — dĺžka (funkcia, nie metóda)

### 25. Aký je rozdiel medzi list a tuple?

**List []** — meniteľný (mutable), môžeš pridávať, mazať, meniť prvky.

**Tuple ()** — nemeniteľný (immutable), po vytvorení sa nedá zmeniť. Rýchlejší, vhodný pre fixné dáta.

### 26. Čo je {"peter": 29, "igor": 40} a ako vypíšem Petrov vek?

Je to slovník (dict) — páry kľúč:hodnota.

```
d = {"peter": 29, "igor": 40}
print(d["peter"]) # 29
```

### 27. Ako pridám prvok do listu? Aké funkcie má list?

`.append(x)` — pridá na koniec  
`.extend([x, y])` — pridá viac prvkov naraz  
`.insert(i, x)` — vloží na pozíciu  
`.remove(x)` — odstráni prvý výskyt hodnoty  
`.pop(i)` — odstráni a vráti prvok (default posledný)  
`.sort()`, `.reverse()`, `.index(x)`, `.count(x)`, `.clear()`

### 28. Ako sa indexuje list? Na čo slúži enumerate()? Ako vypíšem prvý alebo predposledný prvok?

**Indexovanie** — každý prvok má číslo pozície, začína od 0. Záporný index počíta od konca:

```
lst = ['a', 'b', 'c', 'd']
lst[0] # 'a' - prvý
lst[1] # 'b' - druhý
lst[-1] # 'd' - posledný
lst[-2] # 'c' - predposledný
```

**enumerate()** — keď v cykle potrebuješ aj index aj hodnotu zároveň:

```
for i, val in enumerate(['a', 'b', 'c']):
    print(i, val)
# 0 a
# 1 b
# 2 c
```

## 29. Aké funkcie má list?

`.append(x)` — pridá `x` na koniec, nič nevracia

`.remove(x)` — odstráni prvý výskyt `x`, chyba ak neexistuje

`.pop(i)` — odstráni prvok na indexe `i` a vráti ho (default: posledný)

Ďalšie: `.insert()`, `.sort()`, `.reverse()`, `.clear()`, `.copy()`, `.count()`, `.index()`

## 30. Aký je rozdiel medzi `append()` a `extend()` pri listoch?

`.append(x)` — pridá jeden prvok (aj list ako jeden prvok).

`.extend([x, y])` — rozbalí iterable a pridá všetky prvky jednotlivo.

```
a = [1,2]
a.append([3,4])    # [1, 2, [3, 4]]
a = [1,2]
a.extend([3,4])    # [1, 2, 3, 4]
```

## 31. Čo je set? Rozdiel medzi set a tuple?

**set** — neusporiadaná kolekcia unikátnych hodnôt {1,2,3}. Mutable, rýchle hľadanie, bez indexov. Prvky musia byť hashable (nemôžeš tam dať list).

**tuple** — usporiadaná, nemeniteľná sekvencia (1,2,3). Má indexy.  
Set automaticky odstraňuje duplikáty.

## SYNTAX

## 32. Ako vieme escapovať v Pythone?

Spätnou lomkou `\` (`\n` — nový riadok, `\t` — tab)

Alebo raw string (prefix `r`) — ignoruje escape sekvencie:

```
r'C:\Users\meno'
```

## 33. Ako vypíšem premennú v texte?

F-string:

```
print(f"Volám sa {meno} a mám {vek} rokov")
```

## 34. Aký je rozdiel medzi `continue` a `break` v cykle?

**break** — ukončí celý cyklus.

**continue** — preskočí zvyšok aktuálnej iterácie, cyklus pokračuje ďalej.

### 35. Čo je `cislo % 2 == 0`?

Podmienka ktorá zisťuje, či je číslo párne (zvyšok po delení 2 je nula).

### 36. Ako handluje error v Pythone? (try/except/finally)

**try** — sem dáš kód, ktorý môže zlyhať.

**except** — sem dáš, čo sa má stať keď nastane chyba.

**finally** — toto sa vykoná VŽDY, aj keď chyba nastala aj keď nenastala.

```
try:
    x = int(input()) # môže zlyhať
except ValueError:
    print("Nie je číslo") # nastane pri chybe
finally:
    print("Toto vždy") # nastane vždy
```

### 37. Čo znamená type casting?

manuálna konverzia hodnoty z jedného typu na iný.

```
int('42')          # 42
str(42)            # '42'
float('3.14')      # 3.14
list((1,2,3))      # [1, 2, 3]
```

### 38. Ako zistím dátový typ hodnoty v premennej?

**type()** vráti typ.

**isinstance()** je lepšie pre kontroly v kóde — funguje aj s dedičnosťou.

```
type(42)           # <class 'int'>
isinstance(42, int) # True
```

### 39. Rozdiel medzi `==` a `is`?

**==** porovnáva hodnotu.

**is** porovnáva identitu — či sú to ten istý objekt v pamäti.

```
a = [1,2]
b = [1,2]
a == b # True (rovnaká hodnota)
a is b # False (rôzne objekty)
x is None # správny spôsob kontroly None
```

## MODULY

### 40. Na čo slúži regulárny výraz (regex)?

je to taky wild card

textový reťazec, ktorý obsahuje špeciálne znaky. Najčastejšie sa regex využíva pri vyhľadávaní alebo manipulácii s textom.

```
import re
re.findall(r'\d+', 'mám 3 psy a 2 mačky') # ['3', '2']
```

### 41. Na čo slúži venv?

Virtuálne prostredie — izolovaný Python s vlastnými balíčkami pre projekt. Zabraňuje konfliktom verzií medzi projektmi.

```
python -m venv venv
venv\Scripts\activate # Windows
source venv/bin/activate # Linux/Mac
```

### 42. Na čo slúži pip?

Správca balíčkov pre Python. Inštaluje knižnice z PyPI.

```
pip install requests
pip uninstall requests
pip list
pip freeze > requirements.txt
```

### 43. Aké moduly/knižnice v Pythone poznáš?

**Štandardná knižnica:** os, sys, re, math, datetime, json, random, pathlib

**Externé:** requests (HTTP), flask/django (web), pandas (dáta), numpy (numerika), pytest (testy)

### 44. Na čo slúži knižnica requests?

HTTP požiadavky — komunikácia s webom / API.

Podporuje GET, POST, PUT, DELETE, hlavičky, autentifikáciu.

```
import requests
r = requests.get('https://api.github.com')
print(r.status_code) # 200
print(r.json())
```

### 45. Na čo slúži knižnica os? Čo robí os.environ.get()?

**os** — interakcia s OS: súbory, adresáre, env premenné.

**os.environ.get('API\_KEY')** — vráti hodnotu env premennej alebo None ak neexistuje.

```
import os
os.environ.get('API_KEY') # None ak neexistuje
```

## 46. Na čo slúži knižnica sys? Vysvetli sys.argv.

**sys** — prístup k systémovým parametrom interpretera.

**sys.argv** — list argumentov príkazového riadku.

- `sys.argv[0]` = názov skriptu
- `sys.argv[1]` = prvý argument

```
# python script.py hello
# sys.argv = ['script.py', 'hello']
```

## 47. Ako vieme importovať moduly?

Rôzne spôsoby importu:

```
import os                # celý modul
import numpy as np       # pod iným menom
from os import path       # konkrétna vec
from os import *          # všetko (neodp.)
import moj_modul          # vlastný modul
```

## POKROČILÉ

## 48. Aký je rozdiel medzi deepcopy a shallow copy?

**Shallow copy:** kopíruje objekt, ale vnorené objekty sú stále zdieľané (referencie).

**Deep copy:** kopíruje všetko vrátane vnorených objektov — úplne nezávislá kópia.

```
import copy
a = [[1,2],[3,4]]
b = copy.copy(a)      # shallow
```

## 49. čo je referencia ? rozdiel medzi reference a shallow copy ?

**Referencia:** odkaz na ten istý objekt v pamäti

**Shallow copy:** kopíruje objekt, ale vnorené objekty sú stále zdieľané (referencie).

**Rozdiel:** referencia nekopíruje nič, shallow copy kopíruje len prvú úroveň.

## 50. Čo je rekurzia?

Funkcia, ktorá volá samu seba.

Musí mať podmienku ukončenia (base case), inak spadne na RecursionError.

```
def faktorial(n):
    if n <= 1: return 1
    return n * faktorial(n-1)
```

## 51. Ako funguje with statement a context manager?

**with** — zabezpečí, že zdroj sa automaticky uprave po skončení bloku (aj keď nastane chyba).

**Context manager** — objekt, ktorý definuje čo sa stane pri vstupe (`__enter__`) a výstupe (`__exit__`) z with bloku.

```
with open('subor.txt') as f:
    obsah = f.read()
# súbor je automaticky zatvorený
```

## 52. Čo sú closures v Pythone?

Funkcia, ktorá si pamätá premenné z vonkajšieho scope aj po jeho ukončení.

```
def vytvor_counter():
    n = 0
    def counter():
        nonlocal n
        n += 1
        return n
    return counter

c = vytvor_counter()
c() # 1, c() # 2
```

## 53. Aký je rozdiel medzi global a nonlocal?

**global x** — premenná odkazuje na globálnu premennú (úroveň modulu).

**nonlocal x** — premenná odkazuje na premennú z najbližšieho vonkajšieho (nie globálneho) scope. Používa sa v closures.